

BurstOps: Codebase Architecture, Docker Containers & Implementation Guide

Focus: Local Development vs. Azure Cloud Architecture, Docker Containerization, File-by-File Code Tour & Verification Guide

Target Audience: Engineers, Evaluators, Cloud Architects, and Students

Key Philosophy: Complete Local-to-Cloud Parity with Zero Host Pollution

Table of Contents

1. [Why Docker Containers? Benefits & Architectural Rationale](#)
2. [Local Stack vs. Azure Cloud: Architectural Parity](#)
3. [Master Container Inventory: What Every Container Does](#)
4. [Exhaustive Code Tour File-by-File](#)
 - [4.1 The Gateway Engine \(`gateway/`\)](#)
 - [4.2 The Demo Control Backend \(`backend/`\)](#)
 - [4.3 The Azure Serverless Function \(`azure-function/`\)](#)
 - [4.4 Local Stand-Ins \(`dummy-backend/`, `dummy-serverless/`, `cpu-sim/`\)](#)
 - [4.5 Kubernetes Manifests \(`k8s/`\)](#)
 - [4.6 Terraform Infrastructure as Code \(`terraform/`\)](#)
 - [4.7 Frontend Portal \(`frontend/`\)](#)
5. [How to Run, Test, and Verify Locally](#)
6. [Day-2 Operations & Container Debugging](#)

1. Why Docker Containers? Benefits & Architectural Rationale

To build, test, and validate a distributed cloud system with multiple moving parts (gateways, reverse proxies, time-series databases, dashboards, load balancers, and serverless functions), installing tools directly on a developer's host machine is chaotic, error-prone, and unsustainable.

BurstOps uses **Docker containers** across the entire lifecycle for five fundamental engineering reasons:

1.1 Complete Environment Isolation & Dependency Determinism

- **No Python Version Hell:** BurstOps components use specific Python versions (Gateway uses Python 3.12, Azure Function uses Python 3.13, Prometheus runs Go, Grafana runs Go/Node). Containers package the exact Linux runtime, system libraries, and compiled C-extensions in isolated images.
- **Zero Host Pollution:** No packages, databases, or daemons are permanently installed on your laptop. Deleting the project requires only **`docker compose down -v`**.

1.2 Local Kubernetes Cluster Stand-In (At 0 Cloud Spend)

- Running an active Azure Kubernetes Service (AKS) cluster 24/7 costs money (~70/month for VM nodes, load balancers, and egress).
- With Docker Compose, **the local container network acts as a byte-identical replica of a Kubernetes cluster**.
- Containers communicate over Docker's bridge network using DNS service names (**`http://dummy-backend:8000`**, **`http://prometheus:9090`**), replicating Kubernetes ClusterIP and Kube-DNS resolution with zero cloud spend.

1.3 Solves the macOS/Windows cAdvisor Limitation (`cpu-sim`)

- In production Kubernetes on Linux, the kubelet runs **`cAdvisor`**, exposing real per-container CPU metrics (**`container_cpu_usage_seconds_total`**).
- On macOS and Windows, Docker Desktop runs inside a lightweight virtual machine. Host cAdvisor containers cannot inspect individual container cgroups properly.
- We created a dedicated container (**`cpu-sim`**) that publishes PromQL-compatible metrics matching real cAdvisor outputs, enabling full end-to-end testing on developer laptops.

1.4 Immediate Horizontal Scalability Testing

- Testing distributed leader election and state synchronization across multiple gateway replicas is straightforward:

```
docker compose -f docker-compose.yml -f docker-compose.scale.yml up -d --scale gateway=3
```

Docker spins up 3 identical gateway instances, each assigned an ephemeral host port, competing for Redis leases exactly as they do in an AKS cluster.

1.5 100% Portability from Local to Azure Cloud

- The Dockerfile used locally for **gateway** and **dummy-backend** is the exact same Dockerfile built and pushed to **Azure Container Registry (ACR)**.
- What works in Docker Compose is guaranteed to work when scheduled onto AKS worker nodes.

2. Local Stack vs. Azure Cloud: Architectural Parity

BurstOps was engineered so that every single cloud resource has an exact, byte-identical counterpart in the local developer stack:

Component Role	Local Developer Stack (Docker Compose)	Live Microsoft Azure Cloud Stack	Architectural Parity Guarantee
Public Traffic Ingress	Docker Port Mapping (localhost:8080)	Azure Standard Load Balancer (4.224.237.110) + Ingress-NGINX	Both route public HTTP traffic to the active Gateway tier.
Routing Gateway Tier	gateway container (python:3.12-slim , FastAPI)	2x Gateway Pods in AKS namespace burstops-system	Identical source code, PI controller, hysteresis thresholds, and PromQL query.
State Coordination Backplane	redis container (redis:7-alpine)	In-Cluster Redis Deployment in AKS (redis:6379)	Identical Lua atomic leader lease renewal and follower state caching.
Steady-State Workload (K8s)	dummy-backend-1 & dummy-backend-2 behind Nginx VIP (:8000)	Backend Pods behind ClusterIP Service in AKS namespace burstops	Both compute Sieve of Eratosthenes (N=1000), returning 168 primes with sum 76127.
Serverless Burst Target	dummy-serverless container (:8001)	Azure Function App (func-burstops-cloud-ga3cvf) on Flex Consumption	Both enforce identical HMAC-SHA256 signatures, clock skew checks, and return bit-for-bit identical prime payloads.
Metrics Engine	prometheus container (prom/prometheus:v2.53.1)	In-Cluster Prometheus Deployment in AKS (prometheus:9090)	Both scrape cAdvisor and gateway every 2 seconds with 24h retention.
Container Metrics Source	cpu-sim container (:8002) + cadvisor container	Real Kubelet cAdvisor running on AKS Linux Worker Nodes (Standard_B2s_v2)	Both publish container_cpu_usage_seconds_total to Prometheus.
Visualization Dashboard	grafana container (grafana/grafana:11.1.4 , port 3000)	In-Cluster Grafana (/grafana/) + Azure Managed Grafana PaaS	Pre-provisioned with the same 12-panel dashboard and PromQL data queries.
Traffic Load Generator	locust container (locustio/locust:2.29.1 , port 8089)	In-Cluster Locust Deployment in AKS (/locust/)	Both run identical locustfile.py with 20 concurrent users and spawn rate of 5.
Interactive Control Portal	Local Vite Dev Server (localhost:5173)	Render Web Service with single-service static asset hosting	Both provide the full Azure Portal UI with one-click burst and recovery triggers.

3. Master Container Inventory: What Every Container Does

The local BurstOps environment is composed of **10 specialized Docker containers** configured in **docker-compose.yml**, **docker-compose.scale.yml**, and **docker-compose.tracing.yml**:



3.1 gateway — The Core Layer-7 Intelligent Proxy

- **Base Image:** `python:3.12-slim`
- **Exposed Port:** `8080` (or ephemeral host ports in scale mode)
- **Role:**

The brain of the system. Every client request hits this container.

- **How it works:**
 1. A background asynchronous task polls Prometheus every 2 seconds to check cluster CPU utilization.
 2. Runs a **Schmitt-trigger hysteresis state machine**: flips to **BURST** mode when $\text{CPU} \geq 80\%$, returns to **BASELINE** when $\text{CPU} \leq 60\%$.
 3. Executes a continuous **Proportional-Integral (PI) Controller** ($K_p=0.04$, $K_i=0.004$, setpoint 75%) to calculate the deflection ratio $u(t)$ in $[0.0, 1.0]$.
 4. Performs **Bernoulli routing**: if $\text{random float} < u(t)$, deflects request to serverless; otherwise forwards to Kubernetes.
 5. Cryptographically signs serverless requests with **HMAC-SHA256** and injects headers (`x-burstops-signature`, `x-burstops-timestamp`).
 6. Exposes Prometheus metrics on `/metrics` and health/state on `/health`.

3.2 dummy-backend-1 and dummy-backend-2 — The Kubernetes Worker Pods

- **Base Image:** `python:3.12-slim` (FastAPI / Uvicorn)
- **Resource Limits:** `cpus: "1.0"`, `mem_limit: 128m`
- **Role:**

Stand-ins for Kubernetes worker pods processing standard application workload.

- **Workload Canary (Sieve of Eratosthenes):**

Computes all prime numbers up to $N=1000$. The result is deterministic: **168 primes, sum of 76127, largest prime 997**. Any container returning different numbers fails validation.

- **Why Capped at 1.0 CPU?:**

`gateway.py` calculates CPU percentage by taking the rate of cAdvisor CPU seconds times 100. If container CPU was capped too low (e.g. 0.3), the total rate could never reach the 80% threshold. Capping at 1.0 CPU ensures predictable saturation under load.

3.3 dummy-backend (Nginx VIP) — The Kubernetes ClusterIP Service

- **Base Image:** `nginx:1.27-alpine`
- **Exposed Port:** `8000`
- **Role:**

In Kubernetes, pods do not communicate directly with other pod IP addresses; they communicate through a virtual IP (VIP) managed by a Kubernetes **Service**.

This Nginx container serves as that VIP, load-balancing traffic across **dummy-backend-1** and **dummy-backend-2** via round-robin.

3.4 dummy-serverless — The Local Azure Function Clone

- **Base Image:** `python:3.12-slim` (FastAPI / Uvicorn)
- **Exposed Port:** **8001**
- **Role:**

Enables 100% offline development without internet access or Azure credentials.

- **Key Features:**
 1. Enforces the identical **HMAC-SHA256 signature verification** contract using shared secret **GATEWAY_HMAC_SECRET**.
 2. Enforces the **300-second maximum clock skew** window.
 3. Injects simulated serverless cold starts and network latency (50ms–150ms) to reflect real-world cloud conditions.
 4. Returns the identical Sieve of Eratosthenes canary payload (**prime_count: 168, prime_sum: 76127**).

3.5 cpu-sim — Synthetic Prometheus Metric Generator

- **Base Image:** `python:3.12-slim` (FastAPI / Uvicorn)
- **Exposed Port:** **8002**
- **Role:**

Allows manual testing of the state machine without needing real CPU saturation.

- **Endpoints:**
 - **/metrics:** Exposes `container_cpu_usage_seconds_total{namespace="burstops"}` in Prometheus text format.
 - **/ramp-up:** Simulates an abrupt spike (smoothly increases CPU from 20% to 90% over 10 seconds).
 - **/ramp-down:** Simulates recovery (smoothly decreases CPU from 90% to 20%).
 - **/set?pct=X:** Directly pins the simulated CPU to any target value X%.

3.6 cadvisor — Container Advisory Daemon

- **Image:** `gcr.io/cadvisor/cadvisor:v0.49.1`
- **Exposed Port:** **8082**
- **Role:**

Mounts the host Docker socket (`/var/run/docker.sock`) and Linux cgroups (`/sys`) to gather real-time hardware telemetry on memory, CPU, and disk I/O.

3.7 prometheus — Time-Series Telemetry Engine

- **Image:** `prom/prometheus:v2.53.1`
- **Exposed Port:** **9090**
- **Role:**

Scrapes all endpoints configured in `prometheus/prometheus.yml` every 2.0 seconds. Stores time-series data locally with a 24-hour retention window.

3.8 grafana — Operational Visualization Dashboard

- **Image:** `grafana/grafana:11.1.4`
- **Exposed Port:** **3000** (Credentials: `admin` / `burstops`)
- **Role:**

Pre-loaded with datasource configs and a 12-panel dashboard displaying real-time graphs of CPU usage, Gateway routing state, serverless deflection rates, latency percentiles, and FinOps unit costs.

3.9 redis — Distributed Coordination Backplane (Scale Overlay)

- **Image:** `redis:7-alpine`
- **Exposed Port:** **6379**
- **Role:**

Required when running multiple gateway replicas (`docker-compose.scale.yml`). Stores leader leases, broadcast routing modes, and deflection ratios with zero persistence overhead (`allkeys-lru` eviction).

3.10 locust — Workload Stress Generator

- **Image:** `locustio/locust:2.29.1`
- **Exposed Port:** `8089`
- **Role:**

Generates concurrent HTTP traffic against the gateway (`http://gateway:8080/calculate`), simulating flash-crowd spikes to test bursting under load.

4. Exhaustive Code Tour File-by-File

4.1 The Gateway Engine (gateway/)

gateway/gateway.py

The core proxy and decision engine (700+ lines). Key components:

- **RoutingMode(Enum)**: Defines two operational states: **BASELINE = 0** (route to Kubernetes) and **BURST = 1** (route to serverless).
- **PIController**: Implements proportional-integral feedback math:
 - Error term: $e(t) = \text{CPU} - 75.0$
 - Anti-windup clamp: prevents integral accumulation when output saturates at 1.0 or 0.0.
 - Slew-rate limiter: caps rate of change to 0.10/s to prevent oscillations.
- **poll_prometheus_loop()**: Background async task that wakes up every 2 seconds, queries Prometheus via HTTP, updates observed CPU gauge, and evaluates hysteresis thresholds.
- **sign_request_hmac()**: Computes **HMAC-SHA256(secret, timestamp + body)** and injects authentication headers before deflecting to Azure Functions.
- **calculate_proxy()** (**POST /calculate**): The high-throughput request handler. Evaluates Bernoulli random choice against the current deflection ratio, forwards the request using an asynchronous HTTP client pool, and measures request duration in Prometheus histograms.

gateway/gateway_entrypoint.py

The startup wrapper that initializes the production runtime:

- Reads configuration from environment variables (**K8S_UPSTREAM**, **FUNCTION_UPSTREAM**, **GATEWAY_HMAC_SECRET**).
- Detects if **REDIS_URL** is set; if so, activates the distributed state backplane.
- Attaches OpenTelemetry instrumentation if tracing is enabled.

gateway/redis_backplane.py

Implements distributed coordination across multiple gateway replicas:

- **RedisLeaderElector**: Uses an atomic Lua script to acquire a 4-second TTL lease in Redis (**SET burstops:leader <instance_id> NX EX 4**). Renews lease every 1 second.
- **Leader Logic**: Only the elected leader polls Prometheus and calculates the deflection ratio. It writes results to Redis keys: **burstops:routing_mode**, **burstops:deflect_ratio**, **burstops:last_cpu**.
- **Follower Logic**: Non-leader replicas run a 500ms background poller that syncs Redis state into local memory. User requests read from RAM cache with **zero Redis round-trips** on the request path.
- **Failover**: If the leader crashes, its lease expires in 4 seconds; another replica immediately assumes leadership.

gateway/costing.py

FinOps unit economics engine:

- Amortized Kubernetes cost: $C_{k8s} = \backslash 0.000008$ per request.
- Azure Function cost: $C_{serverless} = \backslash 0.000046$ per request.
- Exposes **gateway_cost_usd_total** and **gateway_hpa_lag_cost_usd_total** metrics to Grafana, tracking cloud savings achieved by deflecting only during the HPA provisioning window.

gateway/tracing.py

Distributed tracing integration via OpenTelemetry:

- Injects W3C **traceparent** headers into upstream requests.
- **Adaptive Sampling**: Traces 100% of deflected serverless burst requests (for failure and cold-start diagnostics), but only 5% of steady-state baseline requests to conserve tracing bandwidth.
- **Security Redaction**: Filters out cryptographic secrets and HMAC signatures before exporting spans to Jaeger.

4.2 The Demo Control Backend (backend/)

backend/main.py

FastAPI application serving the control plane and static frontend assets:

- **Endpoints:**
- **GET /api/health:** Health status and target gateway URL.
- **GET /api/demo/status:** Returns current run status, routing mode, and CPU percentage.
- **POST /api/demo/burst:** Initiates Action 1 (Surge Traffic).
- **POST /api/demo/recover:** Initiates Action 2 (Normal Traffic Recovery).
- **GET /api/demo/runs/{id}:** Returns run history and live log stream.
- **Static Asset Serving:** Mounts the built Vite frontend (**frontend/dist**) so the complete portal runs as a single service on Render.

backend/adapters.py

Traffic generation adapters:

- **CloudTrafficAdapter:** Interacts with the live Azure deployment. Signals the in-cluster Locust instance (**/locust/swarm**) and launches concurrent HTTP workers against **/calculate** to trigger bursting.
- **MockTrafficAdapter:** In-memory simulation adapter used for automated testing and offline development without internet access.

backend/manager.py

Coordinates demo run state:

- Manages run lifecycles (**IDLE** → **RAMPING** → **BURSTING** → **RECOVERING**).
- Buffers real-time terminal logs with timestamped log levels (**INFO**, **WARN**, **SUCCESS**).
- Periodically samples both Kubernetes and Azure Function endpoints with Sieve calculations to verify parity.

4.3 The Azure Serverless Function (azure-function/)

azure-function/function_app.py

The cloud serverless target:

- Uses the **Python V2 model** (**azure.functions** decorators).
- **verify_hmac():** Validates **x-burstop-signature** and **x-burstop-timestamp** against **GATEWAY_HMAC_SECRET** using **hmac.compare_digest()** to prevent timing attacks. Enforces 300-second maximum clock skew.
- **sieve_of_eratosthenes():** Executes the prime number canary algorithm, returning 168 primes for N=1000.

azure-function/host.json

Azure Functions host configuration:

- Configures HTTP route prefix to **api** (**routePrefix: "api"**).
- Sets logging levels and application insights telemetry sampling.

4.4 Local Stand-Ins (dummy-backend/, dummy-serverless/, cpu-sim/)

- **dummy-backend/app.py:** Lightweight FastAPI app running the Sieve of Eratosthenes canary algorithm.
- **dummy-backend/nginx.conf:** Configures round-robin load balancing across **dummy-backend-1:8000** and **dummy-backend-2:8000**.
- **dummy-serverless/app.py:** Offline clone of the Azure Function. Implements the same HMAC verification and canary computation with simulated latency.
- **dummy-serverless/hmac_util.py:** Shared HMAC verification logic used by both local mock and test suites.
- **cpu-sim/app.py:** Generates Prometheus metrics on port 8002. Supports **/ramp-up**, **/ramp-down**, and **/set?pct=X**.

4.5 Kubernetes Manifests (k8s/)

The **k8s/** directory contains standard Kubernetes resources for deployment to AKS:

- **namespace.yaml:** Creates isolated namespaces: **burstops** (application workloads) and **burstops-system** (gateway, redis, ingress, monitoring).
- **backend-deployment.yaml & backend-service.yaml:** Deploys backend pods with resource limits and exposes them via a ClusterIP service on port 8000.
- **gateway.yaml:** Deploys BurstOps Gateway pods with environment variables, Prometheus scraping annotations, and readiness probes.

- **redis.yaml**: In-cluster Redis deployment and service for distributed state synchronization.
- **ingress.yaml**: Ingress-NGINX rules routing public traffic to **gateway-service:8080**, **/grafana/**, and **/locust/**.
- **monitoring.yaml**: Prometheus deployment, RBAC permissions, and scrape configs for 2-second cAdvisor and gateway scraping.
- **hpa.yaml**: Configures the Kubernetes Horizontal Pod Autoscaler targeting 70% CPU utilization across backend pods.

4.6 Terraform Infrastructure as Code (terraform/)

The **terraform/** directory manages all Azure cloud resources:

- **main.tf**: Provisions the Resource Group, Storage Account, Log Analytics, Application Insights, Flex Consumption Function App, AKS Cluster, Container Registry, Key Vault, and Azure Managed Grafana.
- **budget.tf**: Creates an automated Azure Consumption Budget alert (100/month) with email notifications to prevent unexpected costs.
- **variables.tf & terraform.tfvars**: Declares deployment parameters (region: **centralindia**, Python version: **3.13**, node VM size: **Standard_B2s_v2**).
- **outputs.tf**: Exports key resource attributes: Function App hostname, AKS cluster name, ACR login server, and Grafana endpoints.

4.7 Frontend Portal (frontend/)

- **frontend/src/App.tsx**: Azure Portal shell layout featuring the top banner, subscription badge, and blade navigation.
- **frontend/src/pages/HomeBlade.tsx**: System architecture diagram and direct links to public cloud endpoints.
- **frontend/src/pages/GatewayBlade.tsx**: Azure Essentials overview, real-time KPI metrics, and Prometheus scrape indicators.
- **frontend/src/pages/LoadTestBlade.tsx**: Action 1 (Trigger Surge) and Action 2 (Restore Baseline) buttons, live terminal log feed, HMAC cryptographic card, and Sieve canary verification card.
- **frontend/src/api.ts**: TypeScript API client communicating with backend endpoints (**/api/demo/burst**, **/api/demo/recover**, **/api/demo/status**).

5. How to Run, Test, and Verify Locally

5.1 Prerequisites

- **Docker & Docker Compose** (Docker Desktop on macOS/Windows or Docker Engine on Linux)
- **Python 3.12 or 3.13**
- **Node.js 18+ and npm** (for frontend development)

5.2 Step 1: Clone the Repository & Configure Environment

```
cd /Users/rajmishara/burstOps

# Copy example environment file
cp .env.example .env
```

5.3 Step 2: Spin Up the Local Docker Stack

```
# Build and start all 10 local containers in the background
docker compose up -d --build

# Verify that all containers are healthy and running
docker compose ps
```

Expected output:

NAME	IMAGE	STATUS	PORTS
burstops-backend-1	burstops-dummy-backend-1	Up	8000/tcp
burstops-backend-2	burstops-dummy-backend-2	Up	8000/tcp
burstops-backend-vip	nginx:1.27-alpine	Up	0.0.0.0:8000->8000/tcp
burstops-cadvisor	cadvisor:v0.49.1	Up	0.0.0.0:8082->8080/tcp
burstops-cpu-sim	burstops-cpu-sim	Up	0.0.0.0:8002->8002/tcp
burstops-gateway	burstops-gateway	Up	0.0.0.0:8080->8080/tcp
burstops-grafana	grafana:11.1.4	Up	0.0.0.0:3000->3000/tcp
burstops-locust	locust:2.29.1	Up	0.0.0.0:8089->8089/tcp
burstops-prometheus	prometheus:v2.53.1	Up	0.0.0.0:9090->9090/tcp
burstops-serverless	burstops-dummy-serverless	Up	0.0.0.0:8001->8001/tcp

5.4 Step 3: Run the Automated Test Suite

The codebase includes 102 comprehensive unit, integration, and security tests covering the gateway, HMAC validation, PI controller, and state machines:

```
# Activate Python virtual environment
source .venv/bin/activate

# Execute all pytest suites
PYTHONPATH=. pytest tests/ -v
```

Expected output:

```
===== 102 passed in 14.82s =====
```

5.5 Step 4: Run the Interactive Web Portal Locally

```
# Terminal 1: Start the Demo Control Backend
source .venv/bin/activate
uvicorn backend.main:app --host 0.0.0.0 --port 8000 --reload

# Terminal 2: Start the React Frontend Dev Server
cd frontend
npm install
npm run dev
```

Open your browser to <http://localhost:5173>. You will see the Azure Portal interface with live controls connected to your local backend.

6. Day-2 Operations & Container Debugging

6.1 Inspecting Live Container Logs

```
# Stream logs from the Gateway container
docker compose logs -f gateway

# Stream logs from the local serverless container
docker compose logs -f dummy-serverless

# Stream logs from Prometheus
docker compose logs -f prometheus
```

6.2 Executing Commands Inside Containers

```
# Open an interactive shell inside the running Gateway container
docker compose exec gateway /bin/bash

# Test local Redis connection (when running scale overlay)
docker compose exec redis redis-cli ping
```

6.3 Manually Simulating CPU Spikes via Terminal

If you want to trigger burst deflection without using the web portal:

```
# 1. Trigger an immediate CPU ramp-up to 90%
curl -X POST http://localhost:8002/ramp-up

# 2. Check Gateway health to verify it entered BURST mode
curl http://localhost:8080/health
# Response: {"mode":"burst","last_cpu":90.0,"deflect_ratio":0.60,...}

# 3. Send a test calculation through the Gateway
curl -X POST http://localhost:8080/calculate -H "Content-Type: application/json" -d '{"max_num":1000}'
# Response shows: "source":"serverless"

# 4. Trigger CPU recovery back to 20%
curl -X POST http://localhost:8002/ramp-down

# 5. Check Gateway health to verify it returned to BASELINE
curl http://localhost:8080/health
# Response: {"mode":"baseline","last_cpu":20.0,"deflect_ratio":0.0,...}
```


6.4 Clean Teardown

To stop all containers, release host ports, and clean up Docker volumes:

```
docker compose down -v
```

This leaves your machine completely clean with zero leftover state.